UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

APUNTES EXTENDIDOS

Sistemas Distribuidos

Clase 08 – Causalidad, consenso y consistencia distribuida

Departamento de Computación

Ingeniería Informática – Comisión A

Pilar – Segundo cuatrimestre 2026

Equipo docente

2026

1 Propósito y pregunta transversal

En esta clase seguiremos el destino de una única entrada VIP que Sofía y Tomás intentan comprar casi al mismo tiempo desde regiones diferentes. Como los nodos no comparten una visión instantánea del sistema ni pueden confiar en un reloj global perfecto, primero utilizaremos causalidad y relojes lógicos para comprender qué eventos están relacionados y cuáles son concurrentes. Luego veremos cómo Raft y Paxos permiten que varias réplicas acuerden una sola decisión autoritativa, y cómo CAP y PACELC obligan a decidir entre esperar, rechazar, responder con información potencialmente atrasada o reparar más tarde. Finalmente, compararemos dos estrategias reales: Dynamo, que favorece disponibilidad conservando versiones y pagando reconciliación, y Spanner, que utiliza consenso y TrueTime para sostener transacciones globales con un orden compatible con lo observado por los usuarios. La pregunta que une toda la clase será: qué error resulta inaceptable para cada operación y qué costo —latencia, indisponibilidad, coordinación o reparación— estamos dispuestos a pagar para evitarlo.

El objetivo no es memorizar una colección de nombres. Para cada mecanismo responderemos:

Pregunta	Qué debe quedar explícito
¿Qué problema aparece?	operación, actores, información disponible y modelo de falla.
¿Qué garantía necesitamos?	observación permitida, invariante o propiedad de progreso.
¿Qué mecanismo la sostiene?	reloj, mayoría, protocolo, versión, reparación o espera.
¿Qué no garantiza?	frontera de la abstracción y supuestos que quedan fuera.
¿Qué cuesta?	latencia, rechazo, metadatos, espacio, operación o complejidad.
¿Cómo lo comprobamos?	timeline, métrica, prueba de falla o contraejemplo.

La unidad de razonamiento será siempre una ejecución concreta. Dibuje primero qué sabe cada nodo y recién después nombre la propiedad o el algoritmo.

2 Método de estudio: Cómo estudiar este capítulo

Use siempre la secuencia problema–falla–garantía–mecanismo–costo–evidencia. Primero reconstruya la ejecución normal; luego mueva un timeout, crash o partición; finalmente indique qué observación permitiría verificar o refutar la explicación. No memorice siglas sin asociarlas con el caso VIP-01.

3 Mapa intuitivo antes del formalismo

Antes de usar ecuaciones o nombres de protocolos, conserve cinco imágenes mentales. No reemplazan la definición técnica: preparan la pregunta que la definición deberá responder.

Idea	Imagen mental	Pregunta que habilita
Falla parcial	Un teléfono que deja de contestar sin revelar si la otra persona está ausente, ocupada o si se cortó la línea.	¿Qué sabe realmente el emisor después de un timeout?

Causalidad	Dos ramas de una conversación que todavía no intercambiaron información.	¿Un evento pudo influir sobre el otro o son concurrentes?
Consenso	Cinco escribanos que deben asentar una única decisión en la misma posición del libro.	¿Qué comando queda autorizado y qué mayoría permite recordarlo?
Dynamo	Varias libretas que aceptan cambios durante el aislamiento y luego comparan genealogías.	¿El conflicto puede conservarse, fusionarse y repararse?
Spanner	Un reloj que no finge exactitud: entrega una ventana donde sabe que se encuentra el tiempo real.	¿Cuánto debe esperar una transacción para respetar el orden externo?

La intuición útil siempre debe terminar en una ejecución verificable. Si la imagen mental no permite ubicar actores, mensajes, estado durable y punto de falla, todavía no alcanza para defender una garantía.

4 Puente desde Kafka

4.1 El ejemplo del Mundial

Un productor observa eventos de un partido y publica registros como `MatchStarted`, `GoalScored`, `YellowCard` o `Substitution`. Kafka los conserva en un log particionado; un consumidor los lee y actualiza el marcador. Cuando hay muchos partidos, una única secuencia concentra red, CPU, almacenamiento y consumo. La solución consiste en particionar por una clave estable, por ejemplo `game_id`.

Si todos los eventos del partido A usan la misma clave, quedan en la misma partición y conservan su orden de append:

$$\texttt{Started(A)} \rightarrow \texttt{Goal(A)} \rightarrow \texttt{YellowCard(A)}.$$

Los partidos A, B y C pueden avanzar en paralelo porque están en particiones distintas. Sin embargo, no existe un offset global gratuito entre esas particiones.

4.2 Consumer groups, replay e idempotencia

Un consumer group distribuye particiones completas entre instancias de la misma aplicación. Una partición tiene una sola instancia activa dentro del grupo en un momento dado. Grupos diferentes pueden recorrer la misma historia con offsets independientes.

Si un consumidor produce un efecto y cae antes de confirmar su progreso, el mismo registro puede reaparecer. Esa semántica *at-least-once* exige que el efecto sea idempotente o que efecto e identidad procesada se registren de manera atómica.

4.3 RF, ISR, minISR y acks

Una configuración frecuente es:

$$RF = 3, \qquad \texttt{min.insync.replicas} = 2, \qquad \texttt{acks=all}.$$

- **RF=3**: la partición tiene un líder y dos seguidores configurados.
- **ISR**: conjunto de réplicas suficientemente sincronizadas con el líder.
- **minISR=2**: con `acks=all`, la escritura solo se acepta si al menos dos réplicas están en ISR.
- **acks=all**: el productor recibe éxito cuando todas las réplicas del ISR actual confirman el batch.

Con dos miembros en ISR se puede seguir produciendo y quedan dos copias sincronizadas. Si solo queda una, la escritura se rechaza en vez de reconocerse con una única copia.

Como guardrails docentes —no como máximos universales— usaremos mensajes habituales menores a 1 MB, alrededor de 1 TB retenido por broker y un presupuesto conservador menor a 10 MB/s de ingreso por broker para ejercicios. La capacidad real depende de tamaño de batch, compresión, almacenamiento, red, número de particiones, replicación y patrón de consumo.

4.4 El límite que motiva la clase

Kafka preserva y distribuye registros. No decide por sí solo cuál de dos comandos incompatibles se convierte en la decisión autoritativa sobre un recurso indivisible. Si Buenos Aires y Madrid intentan vender la única `VIP-01`, conservar ambas solicitudes no alcanza: el sistema debe seleccionar una única transición válida.

5 Por qué importa en sistemas reales

5.1 Kubernetes y etcd

Kubernetes administra el estado deseado y observado de un cluster. El API server utiliza etcd como almacén consistente de ese estado. etcd replica un log mediante Raft: un líder propone cambios, una mayoría los compromete y cada miembro aplica el mismo orden a su máquina de estados clave–valor.

5.2 Amazon Dynamo

Dynamo fue un almacén clave–valor diseñado para servicios que priorizaban continuar respondiendo. Bajo determinadas fallas podía conservar ramas concurrentes y resolverlas posteriormente. El carrito de compras ilustra un conflicto que puede fusionarse: agregar un libro en una región y auriculares en otra puede terminar en un carrito con ambos ítems.

5.3 Google Spanner

Spanner es una base de datos distribuida globalmente. Divide los datos en grupos replicados, usa consenso dentro de esos grupos y coordina transacciones que abarcan varios grupos. TrueTime expone un intervalo de incertidumbre temporal, lo que permite pagar una espera explícita para ofrecer consistencia externa.

6 Modelo mínimo de réplica, red y falla

Una réplica combina:

- un proceso que ejecuta código;

- estado temporal en RAM;
- estado persistente recuperable después de un reinicio;
- una interfaz de red para enviar y recibir mensajes.

Debemos distinguir tres situaciones:

1. **sana y comunicada**: procesa y recibe mensajes dentro del plazo esperado;
2. **sana pero aislada**: sigue ejecutando localmente, pero no puede coordinar con otro grupo;
3. **caída o pausada**: deja de ejecutar pasos durante cierto período.

Un timeout solo prueba que el emisor no recibió una respuesta antes de su plazo. Es compatible con varias historias: el request no llegó, el receptor está lento o pausado, la operación se ejecutó y la respuesta se perdió, o la red está particionada. Esta indistinguibilidad explica por qué aparecen IDs idempotentes, términos, mayorías, fencing y logs durables.

7 Caso conductor: una entrada y cinco réplicas

El recurso es `VIP-01` y el estado inicial es `stock=1`. Sofía compra desde Buenos Aires y Tomás desde Madrid. Las réplicas son $N_1, \ldots, N_5$. El invariante es:

$$\text{reservas confirmadas para VIP-01} \leq 1.$$

La dificultad no es almacenar dos requests. La dificultad es garantizar que solo una transición autoritativa cambie la máquina de estados de `AVAILABLE` a un único propietario.

8 Causalidad y concurrencia

8.1 Happened-before

La relación $\rightarrow$ se define mediante tres reglas:

1. si dos eventos ocurren en el mismo proceso, su orden local induce causalidad;
2. el envío de un mensaje ocurre antes que su recepción;
3. la relación es transitiva.

Si a_1 es el click de Sofía, a_2 el envío desde Buenos Aires y l_1 la recepción en el líder:

$$a_1 \rightarrow a_2 \rightarrow l_1.$$

De modo análogo, $m_1 \rightarrow m_2 \rightarrow l_3$ para Tomás. Antes de que exista un mensaje que conecte ambas ramas, a_2 y m_2 son concurrentes:

$$a_2 \parallel m_2.$$

Concurrencia no significa simultaneidad física exacta; significa que ninguna rama contiene evidencia causal de la otra.

8.2 Relojes de Lamport

Cada proceso mantiene un contador lógico L. Antes de un evento local lo incrementa; al enviar un mensaje adjunta $t_m = L$; al recibir actualiza:

$$L \leftarrow \max(L, t_m) + 1.$$

El máximo conserva tanto la historia local como la comunicada; el incremento coloca la recepción estrictamente después del envío.

Una ejecución posible es:

Proceso	Evento	Lamport
BA	click Sofía	1
BA	envía reserva	2
Madrid	click Tomás	1
Madrid	envía reserva	2
Líder	recibe Sofía	3
Líder	append local	4
Líder	recibe Tomás	5

La propiedad es:

$$a \to b \Longrightarrow L(a) < L(b).$$

El recíproco no vale. Que una etiqueta sea menor no demuestra causalidad; puede ser solo consecuencia de contadores independientes o de una regla de desempate.

8.3 Relojes vectoriales

Un vector mantiene un componente por actor. Con el orden $[\text{BA}, \text{Madrid}, \text{Líder}]$:

$$V(a_2) = [2, 0, 0], \qquad V(m_2) = [0, 2, 0].$$

Ningún vector domina al otro, por lo que son concurrentes. Al recibir a Sofía, hacer append y luego recibir a Tomás, el líder obtiene:

$$[2, 0, 1] \to [2, 0, 2] \to [2, 2, 3].$$

El tercer componente vale 3 porque el líder ejecutó tres eventos locales. El valor 5 pertenece al reloj escalar de Lamport y no debe copiarse dentro del vector.

Lamport permite producir etiquetas compatibles con causalidad; un vector permite distinguir posterioridad de concurrencia; ninguno decide quién obtiene la entrada.

9 Máquinas de estados y consenso

9.1 Máquina de estados determinista

Una máquina de estados aplica una función:

$$(estado, comando) \longrightarrow (nuevo\ estado, respuesta).$$

Para la entrada:

$$\texttt{AVAILABLE} + \texttt{Reserve(Sofía)} \rightarrow \texttt{RESERVED_BY_SOFIA}.$$

Una nueva reserva sobre un estado ya reservado devuelve rechazo sin crear un segundo propietario. Si varias réplicas aplican los mismos comandos en el mismo orden, convergen en el mismo estado.

9.2 Safety y liveness

Safety significa que nada incorrecto ocurre: dos réplicas correctas no comprometen comandos incompatibles para el mismo índice, una entrada committed no se reemplaza y `VIP-01` no termina confirmada para dos personas.

Liveness significa que el sistema vuelve a progresar cuando existen las condiciones necesarias: una mayoría comunicada, un líder estable, procesos que continúan ejecutando pasos y entrega eventual de los mensajes requeridos. Un protocolo puede preservar safety durante una partición permanente y perder liveness en la minoría.

10 Raft paso a paso

10.1 Término, índice y entrada

Un término es un número creciente que identifica una ronda de elección y liderazgo. Un índice identifica una posición del log. La entrada didáctica:

$$(\mathit{index} = 87,\ \mathit{term} = 12,\ \mathit{command} = \texttt{Reserve(VIP-01,Sofía)})$$

usa 87 porque las posiciones 1–86 ya estaban ocupadas y 12 porque el cluster se encuentra en su duodécima ronda electoral. Un mensaje con término menor al conocido por el receptor es obsoleto y se rechaza.

10.2 Replicación y commit

1. N_1, líder, agrega la entrada localmente. Aún no es una decisión committed.
2. N_2 persiste la entrada. Dos copias de cinco todavía no forman mayoría.
3. N_3 persiste la entrada. Tres de cinco forman mayoría.
4. N_1 actualiza `commitIndex=87`, aplica el comando en su máquina local y responde a Sofía.
5. N_2 y N_3 reciben el nuevo `leaderCommit`, avanzan su commitIndex y aplican localmente.

Deben distinguirse estos estados:

$$recibida \neq agregada \neq replicada \neq committed \neq aplicada \neq confirmada.$$

10.3 Mover el crash

Si N_1 cae cuando solo él posee la entrada, esta no estaba committed y puede desaparecer al elegir un nuevo líder; Sofía no debe haber recibido confirmación. Si cae después de que N_1, N_2, N_3 la persistieron y comprometieron, una mayoría futura intersecta con esa decisión y la conserva.

10.4 Total-order broadcast

Las máquinas deterministas convergen si reciben los mismos comandos committed en el mismo orden. Acordar un valor para el índice 87, luego para 88, 89 y así sucesivamente construye un log con orden total. Un timestamp Lamport no basta: etiqueta eventos, pero no garantiza que todas las réplicas entreguen el mismo conjunto sin huecos.

10.5 Pausas, GC y fencing

Una pausa de garbage collection puede hacer que un líder deje de emitir heartbeats sin estar muerto. Otro nodo puede ser elegido en un término superior. Cuando el líder antiguo despierta, sus mensajes internos se rechazan por término obsoleto.

Un recurso externo que no conoce los términos de Raft puede necesitar un **fencing token**: número creciente que acompaña cada operación y permite rechazar órdenes de una autoridad antigua.

10.6 Paxos y Multi-Paxos

Paxos básico acuerda un valor; para construir un log se repite por slot. Multi-Paxos estabiliza un proposer/líder para muchos slots. En la misma posición 87, Raft habla de término, índice y followers; Multi-Paxos de ballot, slot y acceptors. La meta común es impedir que dos valores incompatibles queden elegidos para la misma posición.

11 Tres logs que no deben confundirse

Log	Pregunta que responde
Log de consenso	¿Qué comandos fueron acordados y en qué orden autoritativo?
WAL local	¿Qué puede reconstruir cada nodo después de un crash?
Log de eventos	¿Qué hechos pueden leer y reprocesar otros componentes?

Que una entrada esté committed en Raft no elimina la necesidad de persistencia local correcta; publicar un evento no sustituye el acuerdo sobre el estado autoritativo.

12 CAP, disponibilidad y PACELC

12.1 Dos historias indistinguibles

Supongamos que N_1, N_2, N_3 comprometieron la reserva de Sofía y la red se divide entre $\{N_1, N_2, N_3\}$ y $\{N_4, N_5\}$. Tomás llega a la minoría. Para N_4, N_5 son compatibles dos historias:

- Sofía ya fue confirmada, pero el mensaje no cruza;

- Sofía nunca fue confirmada.

Si la minoría debe responder éxito en la segunda historia para permanecer disponible, dará la misma respuesta en la primera y puede vender dos veces. Si espera o rechaza, preserva el invariante y sacrifica disponibilidad CAP para esa operación.

12.2 Uptime y disponibilidad CAP

Uptime es la fracción de una ventana durante la cual un proceso o endpoint estuvo levantado y alcanzable. La ventana puede ser mensual o anual por decisión de medición.

Disponibilidad CAP exige que una solicitud recibida por un nodo no fallado termine con una respuesta no errónea. Un 503 `sin quorum` rápido puede demostrar que el proceso está vivo, pero no cumple disponibilidad CAP para la compra.

Disponibilidad semántica mide cuántas operaciones cumplen la promesa de negocio. Es otra métrica y debe definirse por dominio.

12.3 PACELC

CAP estudia la decisión durante una partición. PACELC agrega la ruta normal: incluso sin partición, coordinar regiones remotas aumenta latencia y dependencia. Una lectura local de 15 ms puede estar atrasada; una lectura coordinada de 180 ms puede ofrecer una garantía más fuerte. El dominio decide cuándo paga esa diferencia.

13 Recencia, linearizabilidad y serializabilidad

Recencia describe cuán nuevo es el estado observado; puede medirse como antigüedad o lag.

Linearizabilidad exige que cada operación parezca ocurrir atómicamente entre su invocación y respuesta y respete el orden real de operaciones no superpuestas. Si una escritura terminó antes de comenzar una lectura, esa lectura debe verla o ver algo posterior.

Serializabilidad exige que el resultado de varias transacciones sea equivalente a algún orden serial, pero no necesariamente al orden temporal externo.

Consistencia externa combina un orden serial con respeto del orden observado por clientes fuera del motor.

14 Mayoría de consenso y quórums Dynamo-like

En consenso, una mayoría participa en una prueba de safety que elige una única entrada autoritativa para un índice. En un sistema Dynamo-like, N, R, W describen cuántas réplicas preferidas existen y cuántas respuestas completan lecturas y escrituras de una clave.

Si:

$$R + W > N,$$

cualquier conjunto ideal de lectura y escritura de esos tamaños intersecta. La intersección no crea por sí sola un líder, un orden total ni linearizabilidad.

14.1 Tres contraejemplos

1. **Escrituras concurrentes:** el nodo de intersección puede conservar dos versiones hermanas; sabe que hay conflicto, pero no inventa un orden.
2. **Sloppy quorum:** la escritura efectiva puede realizarse en nodos alternativos fuera del conjunto preferido; una lectura temprana sobre propietarios puede no cruzar ese conjunto antes del handoff.
3. **Last-write-wins:** un reloj adelantado puede hacer que una actualización causalmente anterior parezca más nueva y descarte información válida.

15 Version vectors y conflictos fusionables

Un version vector registra conocimiento causal por actor. Una versión domina a otra si todos sus componentes son mayores o iguales y al menos uno es mayor. Si ninguna domina, son concurrentes.

Para el carrito:

$$v_{BA} = [1, 0] \quad \text{libro}, \qquad v_{MAD} = [0, 1] \quad \text{auriculares}.$$

Las ramas pueden fusionarse por una regla de dominio. Para `VIP-01`, fusionar `RESERVED_BY_SOFIA` con `RESERVED_BY_TOMAS` violaría el invariante. Detectar concurrencia no implica que exista un merge válido.

16 Dynamo: disponibilidad y deuda de reparación

El flujo conceptual es:

1. calcular el hash de la key y ubicar su rango;
2. construir la preference list de réplicas;
3. reunir W respuestas para aceptar una escritura;
4. conservar versiones concurrentes cuando no puede ordenarlas;
5. usar sloppy quorum si propietarios preferidos están inaccesibles;
6. guardar un hint que indica el propietario final;
7. ejecutar hinted handoff cuando el propietario regresa;
8. usar read repair y anti-entropy para corregir divergencias restantes.

Hinted handoff reubica escrituras desplazadas. **Read repair** corrige réplicas atrasadas descubiertas durante una lectura. **Anti-entropy** compara rangos en background para encontrar diferencias, incluso en datos fríos.

Podemos pensar la deuda mediante:

$$D_{nuevo} > R_{resuelto} \Longrightarrow \text{divergencia acumulada creciente.}$$

Los costos son versiones hermanas, hints pendientes, espacio adicional, red e I/O de reparación, frescura variable y semántica de merge en la aplicación.

17 Spanner y TrueTime

17.1 Por qué aparece en la historia

Supongamos que una compra debe modificar el grupo de entradas, el grupo de pagos y el grupo de derechos del usuario. Además, una operación en Madrid comienza después de que un cliente observó la confirmación en Buenos Aires. Ya no alcanza con consistencia eventual: necesitamos una transacción multi-grupo y un orden compatible con el tiempo real observado.

17.2 Consenso localizado

Spanner divide datos en grupos o rangos replicados. Cada grupo usa consenso para mantener su estado. Una transacción que abarca varios grupos coordina solamente esos participantes; no existe una única votación planetaria para todas las operaciones.

17.3 TrueTime

TrueTime devuelve un intervalo:

$$TT.now() = [earliest, latest]$$

que declara una cota de incertidumbre. Los servidores se sincronizan con referencias y time masters; la amplitud refleja error de sincronización, deriva y tiempo desde la última sincronización. Conceptualmente, si la estimación es t y la cota es ϵ:

$$earliest = t - \epsilon, \qquad latest = t + \epsilon.$$

17.4 Timestamp de commit y commit wait

El coordinador de la transacción elige un timestamp s compatible con los grupos participantes y la cota temporal. Después espera hasta poder demostrar:

$$TT.now().earliest > s.$$

En ese momento, s quedó definitivamente en el pasado. La espera convierte una cota explícita de incertidumbre en una garantía de orden externo.

17.5 Consistencia externa

Se llama externa porque respeta el orden observado por clientes fuera del motor. Si el cliente recibe el commit de T_1 y después inicia T_2, toda serialización válida debe cumplir:

$$T_1 < T_2.$$

18 Mapa final: qué pregunta responde cada mecanismo

Mecanismo	Pregunta

Kafka	¿Qué registros pueden preservarse, particionarse y releerse?
Happened-before	¿Qué evento pudo influir sobre otro?
Lamport	¿Cómo etiquetar eventos respetando causalidad?
Vector clocks	¿Una versión es posterior o concurrente?
Raft/Paxos	¿Qué comando se convierte en decisión autoritativa?
WAL local	¿Qué recuerda cada nodo después de reiniciar?
CAP	¿Qué se sacrifica durante una partición?
PACELC	¿Qué costo pagamos normalmente por coordinar?
Quórums Dynamo	¿Con cuántas réplicas completamos una operación?
Dynamo	¿Cómo seguir respondiendo y reparar divergencia?
Spanner/TrueTime	¿Cómo sostener transacciones globales y orden externo?

19 Síntesis final

La clase puede resumirse como una cadena de preguntas. Kafka preserva registros y orden local, pero no elige por sí solo una transición autoritativa sobre un recurso indivisible. La causalidad y los relojes lógicos describen relaciones y conocimiento, pero no sustituyen un protocolo de acuerdo. Raft y Paxos convierten propuestas en una secuencia comprometida; el WAL local permite que cada réplica recuerde lo prometido. CAP obliga a decidir qué hacer durante una partición y PACELC recuerda que coordinar también cuesta cuando la red funciona. Dynamo acepta versiones y deuda de reparación cuando el dominio admite reconciliación. Spanner paga consenso, infraestructura temporal y espera para sostener transacciones globales con orden externo.

El criterio transversal es el mismo: identificar el error intolerable, declarar la falla, ubicar la frontera de confirmación y medir el costo de la garantía elegida. Ningún nombre de producto reemplaza esa argumentación.

20 Errores frecuentes y cómo corregirlos

Error	**Corrección operativa**
"Timeout significa que el servidor cayó."	Construir al menos cuatro historias compatibles y buscar evidencia durable o una consulta idempotente.
"Lamport menor implica causa."	Dibujar dos procesos sin mensajes: las etiquetas pueden ordenarse sin relación causal.
"Tres copias significan commit."	Exigir término, índice, regla de mayoría y relación con mayorías futuras.
"Un rechazo rápido es disponibilidad CAP."	Separar uptime del proceso de una respuesta válida de la operación solicitada.
"Si $R+W>N$, la lectura es linearizable."	Probar escrituras concurrentes, sloppy quorum y una política de reconciliación insuficiente.
"Eventual significa que se arregla solo."	Pedir tasa de divergencia, capacidad de repair, edad máxima y SLO de convergencia.
"TrueTime elimina la incertidumbre."	Mostrar el intervalo $[earliest, latest]$ y hacer crecer su amplitud.

21 Glosario operativo

ACK Respuesta que declara cumplida una condición del protocolo; debe decir qué condición.

Commit Frontera a partir de la cual una decisión tiene la propiedad declarada por el sistema.

Concurrente Sin relación causal en ninguno de los dos sentidos.

Fencing token Número creciente que permite rechazar a una autoridad antigua.

ISR Réplicas Kafka suficientemente sincronizadas con el líder.

Linearizabilidad Una sola copia aparente compatible con tiempo real de operaciones no superpuestas.

Quorum Conjunto de participantes cuya respuesta permite completar una etapa definida.

Safety Nada incorrecto ocurre.

Liveness El sistema progresa bajo condiciones especificadas.

Sloppy quorum Operación aceptada temporalmente en nodos alternativos.

TrueTime API que devuelve un intervalo acotado de tiempo, no un instante perfecto.

22 Autoevaluación

1. ¿Qué demuestra un timeout y qué no demuestra?
2. ¿Por qué $L(a) < L(b)$ no prueba $a \to b$?
3. ¿Cómo demuestra un vector que dos eventos son concurrentes?
4. ¿Por qué $[2, 2, 3]$ es correcto para el líder?
5. ¿Qué diferencia existe entre replicated, committed y applied en Raft?
6. ¿Qué falla tolera una mayoría de cinco y qué ocurre en la minoría?
7. ¿Por qué total-order broadcast no se obtiene solo con timestamps?
8. ¿Qué protege un término y qué protege un fencing token?
9. ¿En qué sentido Paxos y Raft resuelven el mismo problema?
10. ¿Por qué un 503 rápido no es disponibilidad CAP?
11. ¿Qué agrega PACELC?
12. ¿Qué garantiza $R + W > N$ y qué no?
13. ¿Cómo produce un sloppy quorum un contraejemplo a una lectura fresca?
14. ¿Cuándo un conflicto es fusionable?
15. ¿Qué deuda crea Dynamo al seguir escribiendo durante fallas?
16. ¿Qué mide la tasa de divergencia?
17. ¿Por qué TrueTime usa un intervalo?
18. ¿Quién elige s y qué espera commit wait?
19. ¿Por qué la consistencia de Spanner se llama externa?

20. Para ranking, carrito, ledger y entrada VIP: ¿qué error es inaceptable en cada caso?

23 Trabajo Final Integrador

Para cada subsistema del Trabajo Final Integrador, complete una ficha con el marco

$$D = (P, F, G, S, T),$$

donde P es el problema, F las fallas consideradas, G las garantías observables, S la solución y T los trade-offs. La ficha debe identificar además:

- la unidad de autoridad o el dato cuyo invariante se protege;
- la clave, partición o grupo de consenso relevante;
- el evento exacto después del cual se responde al cliente;
- el estado que debe sobrevivir a un reinicio;
- el comportamiento durante una partición;
- una métrica de costo normal y otra de recuperación;
- una prueba adversarial capaz de refutar el diseño.

No se aceptan frases como “usaremos consenso” o “la consistencia será eventual” sin indicar operación, frontera y condición de éxito. Dos subsistemas del mismo proyecto pueden justificar políticas distintas.

24 Bibliografía guiada

- Martin Kleppmann, *Designing Data-Intensive Applications*, capítulos 5, 8 y 9.
- Leslie Lamport, *Time, Clocks, and the Ordering of Events in a Distributed System*.
- Diego Ongaro y John Ousterhout, *In Search of an Understandable Consensus Algorithm*.
- Leslie Lamport, *Paxos Made Simple*.
- Giuseppe DeCandia et al., *Dynamo: Amazon's Highly Available Key-value Store*.
- James Corbett et al., *Spanner: Google's Globally-Distributed Database*.